

CM Track 22: Der Miet-Marktplatz — Inserat, Anfrage, Buchung, Zahlung, Umsatz, Bewertung

Datum: 2026-08-28 **Stack:** Next.js 15, React 19, TypeScript 5.9, Supabase SDK 2.98, Stripe 20.4
Ausgangsstand: 2737dde (Track 21) **Tests:** 1508 → 1526 (18 neue, 2 angepasste), alle grün
Typecheck: `tsc --noEmit` ohne Fehler · **Lint:** 0 Fehler, 21 Warnungen (keine davon in einer Datei dieses Tracks)

Zusammenfassung

Geprüft wurde die **zweite Geldstrecke der Plattform**: die Stuhlmiete. Also alles zwischen dem Moment, in dem ein Vermieter ein Inserat anlegt, und dem Moment, in dem der Payout-Cron sein Geld auf sein Connect-Konto überweist — Inserate (`rental_equipment`), Anfragen (`rental_requests`), Buchungen (`rental_bookings`), die Miet-Zweige des Stripe-Webhooks, die Provision, die Umsatzanzeige des Vermieters und die bidirektionalen Miet-Bewertungen.

Diese Domäne war bis hierher nur punktuell berührt: Track 7 hat die Sichtbarkeit der Inserate repariert, Track 12 den Miet-Storno nachgeliefert, Track 15 die Anbieter-Sperre auf die Miet-Geldstrecken gelegt, Track 18 den Berliner Kalendertag in der Buchung. Sie war nie **als Kette** angesehen worden.

Sieben Befunde, sechs davon im Code behoben und deployed, einer als committete Migration. Zwei Linien ziehen sich durch fast alle:

1. **Was für den Termin gilt, galt für die Miete nicht.** Der Termin-Zweig und der Bestell-Zweig des Stripe-Webhooks haben seit Track 16 einen Compare-and-Swap auf dem Übergang `unpaid → paid`. Der Miet-Zweig war der einzige ohne. Der Datumsvergleich der Buchung läuft seit Track 18 auf dem Berliner Kalendertag — der der Anfrage lief unverändert in UTC weiter. Beides sind keine neuen Fehler, sondern **Reparaturen, die eine Strecke ausgelassen haben**.

2. **Funktionen, die eine Schemaänderung nicht mitbekommen haben.** Der schwerste Befund ist ein direkter Verwandter von CM21-06: eine Stored Procedure, die eine Buchung in der Tabelle sucht, in der sie seit Mai nicht mehr steht — und die deshalb still nichts tut, ohne einen Fehler zu melden.

Die beiden schwersten:

1. **Die 14-Tage-Freischaltung der Miet-Bewertungen ist nie gelaufen.** `publish_review_pair()` löst das Buchungsende ausschließlich aus `public.bookings` auf. Miet-Bewertungen tragen dort seit Migration `20260702_reviews_rental_bookings` **absichtlich** eine `rental_bookings.id`. Für jede einzelne von ihnen findet die Funktion keine Zeile, rechnet mit `NULL` und schaltet nichts frei. Die Oberfläche verspricht dabei wörtlich „spätestens nach 14 Tagen“. Der nächtliche Cron hat den Aufruf als Erfolg gezählt, weil er wirklich gelingt — er tut nur nichts.

2. **Unbezahlte Reservierungen zählten als Einnahme.** `/api/me/rental-revenue` markierte eine Buchung im Zustand `pending` / `unpaid` als Umsatz, und die Seite `/vermieter/mein-inserat/umsatz` filtert genau auf dieses Feld, summiert daraus „Einnahmen gesamt“, rechnet die Auslastung und leitet

daraus eine Preisempfehlung ab. Solche Zeilen kostet das Anlegen nichts: die Buchung entsteht **vor** dem Stripe-Checkout. Ein beliebiges angemeldetes Konto konnte damit die Umsatzkurve eines fremden Vermieters bestimmen — auf derselben Seite, deren Leerzustand verspricht: „Sobald die erste Buchung *bezahlt* ist, erscheinen hier echte Zahlen.“

Ohne Befund geblieben ist die eigentliche Preisstrecke: der Mietpreis wird ausnahmslos serverseitig aus `rental_equipment` gerechnet, der Client schickt nur Zeitraum und Objekt-ID; die Provisionsberechnung, der Escrow bis zum Mietbeginn, die Doppelzahlungs- und Overlap-Verteidigung im Webhook und die Auszahlungsprüfungen des Payout-Crons (Teilerstattung, Rückbuchung, mehrere Connect-Konten) sind alle sauber. Details unter „Ohne Befund“.

Befundübersicht

ID	Befund	Schwere	Status
CM22-01	<code>publish_review_pair()</code> sucht Miet-Buchungen in der falschen Tabelle — 14-Tage-Freischaltung nie gelaufen	HOCH (P1)	behoben (Code) + Migration
CM22-02	Miet-Zweig des Stripe-Webhooks ohne Compare-and-Swap	MITTEL (P2)	behoben
CM22-03	<code>rental_equipment</code> mit dem öffentlichen ANON-Key vollständig lesbar	MITTEL (P2)	Migration committet, NICHT angewendet
CM22-04	Unbezahlte Reservierungen zählten als Umsatz des Vermieters	MITTEL (P2)	behoben
CM22-05	<code>/api/rental-requests</code> ohne eigenes Limit — Postfach und Mailbox des Vermieters flutbar	MITTEL (P2)	behoben
CM22-06	Fälligkeitsvergleich der Anfrage und des Payout-Crons in UTC statt Berlin	NIEDRIG (P3)	behoben
CM22-07	Datumsangaben nur auf ihre Form geprüft — <code>2026-13-45</code> lief als <code>NaN</code> durch beide Riegel	NIEDRIG (P3)	behoben

Dazu vier CHECK-Constraints in derselben Migration, die Regeln festschreiben, die heute nur im Route-Handler stehen (siehe „Migration“).

CM22-01: Die 14-Tage-Freischaltung der Miet-Bewertungen ist nie gelaufen (HOCH)

Was versprochen wird

`src/app/api/reviews/rental/route.ts` antwortet auf jede abgegebene Bewertung mit einer festen Zusage:

Deine Bewertung wird sichtbar, sobald beide Seiten bewertet haben (spätestens nach 14 Tagen).

Das ist das Double-Blind-Modell: beide Seiten bewerten verdeckt, damit niemand aus Rache antwortet. Wenn nur eine Seite bewertet, wird ihre Bewertung nach 14 Tagen trotzdem sichtbar —

sonst könnte man eine schlechte Bewertung dadurch verhindern, dass man selbst keine abgibt.

Beweis

Die Freischaltung liegt in der Stored Procedure `publish_review_pair()` (`supabase/migrations/20260515_bidirectional_reviews.sql`). Sie löst das Buchungsende so auf:

```
SELECT COALESCE(end_at, updated_at, created_at) INTO v_booking_ended_at
FROM public.bookings
WHERE id = p_booking_id;

v_days_since_booking := EXTRACT(DAY FROM NOW() - v_booking_ended_at);
...
IF v_days_since_booking >= 14 THEN -- Fall 2
```

Zwei Monate später hat `supabase/migrations/20260702_reviews_rental_bookings.sql` den Fremdschlüssel entfernt — **mit Ansage**:

```
-- Bidirektionale Miet-Bewertungen (tenant_to_provider / provider_to_tenant)
-- referenzieren rental_bookings.id – nicht bookings.id.
-- Ein evtl. vorhandener FK auf bookings(id) würde diese Inserts blocken.
ALTER TABLE public.reviews DROP CONSTRAINT IF EXISTS reviews_booking_id_fkey;
```

`reviews.booking_id` ist seitdem polymorph. Die Funktion wurde nicht mitgezogen. Für jede Miet-Bewertung gilt deshalb:

- der `SELECT` findet keine Zeile → `v_booking_ended_at` bleibt `NULL`
- `EXTRACT(DAY FROM NOW() - NULL)` → `NULL`
- `IF NULL >= 14 THEN` ist **nicht wahr** → Fall 2 wird nie betreten

Fall 1 (beide Seiten haben bewertet) braucht das Buchungsende nicht und funktionierte weiter. Genau deshalb ist der Defekt nie aufgefallen: die Freischaltung *passierte* manchmal, nur eben nie ohne Gegenbewertung.

Sichtbar war es auch sonst nicht. Die Funktion gibt `VOID` zurück und meldet keinen Fehler, wenn sie nichts findet. `supabase.rpc()` lieferte also `error: null`, und der nächtliche Cron zählte `published++`:

```
const { error } = await supabase.rpc('publish_review_pair', { p_booking_id: bookingId })
if (error) { failed.push(bookingId); continue }
published++ // ← der Aufruf gelingt. Er tut nur nichts.
```

Track 20 hat an genau dieser Stelle das Fehler-Handling repariert (`rpc()` wirft nicht, der Fehler steht im Rückgabewert). Die Meldung blieb trotzdem falsch — weil hier kein Fehler auftritt.

Folgen: eine einseitige Miet-Bewertung — der Normalfall — bleibt unbegrenzt als Entwurf liegen. `profiles.avg_rating_as_tenant` und `avg_rating_as_provider` werden nur vom Trigger `trg_update_user_review_aggregates` gefüllt, und der zählt ausschließlich `published = TRUE`. Die gesamte Mieter- und Vermieter-Reputation existiert also nur für Buchungen, bei denen beide Seiten

bewertet haben. Zweiter Effekt: der Cron liest jede Nacht dieselben 500 ältesten Entwürfe, ohne je einen davon abzuräumen — neu fällig werdende Bewertungen können aus dem Limit fallen.

Fix

Zwei Schichten, weil nur eine davon ohne Migrationslauf wirkt.

Anwendungsseitig (deployed): `/api/cron/publish-reviews` schaltet die fälligen Bewertungen selbst frei, statt sich auf eine Funktion zu verlassen, die in der falschen Tabelle sucht:

```
const { data: updated, error } = await supabase
  .from('reviews')
  .update({ published: true, visible_at: new Date().toISOString() })
  .in('id', ids)
  .eq('published', false) // Claim gegen den Pfad beim Absenden
  .select('id')           // ← die Zahl unten meldet Geschriebenes, nicht Versuchtes
```

Dass `created_at` als Fälligkeitsmaß genügt, folgt aus der Route selbst: eine Miet-Bewertung kann erst **nach** dem Ende der Buchung abgegeben werden (`bookingEnded()`). `created_at >= Buchungsende` gilt also immer, und „`created_at` älter als 14 Tage“ ist damit eine konservative Näherung — sie schaltet nie zu früh frei, höchstens später.

Datenbankseitig (Migration, committet): `publish_review_pair()` löst das Buchungsende jetzt polymorph auf — erst `bookings`, dann `rental_bookings` — und findet ohne Ergebnis gar keins, statt mit `NULL` weiterzurechnen. Damit wirkt die 14-Tage-Regel auch auf dem Pfad, der direkt nach dem Absenden läuft.

CM22-02: Der Miet-Zweig des Webhooks war der einzige ohne Compare-and-Swap (MITTEL)

Beweis

Alle drei Zahlungsarten laufen durch dieselbe Datei. Zwei von ihnen sichern den Übergang `unpaid` → `paid` mit einem Claim ab, seit Track 16, und der Kommentar darüber sagt auch warum:

```
// fulfillBookingPayment (Zeile 279 ff.) – und fulfillProductOrder identisch
.eq('id', bookingId)
.neq('payment_status', 'paid') // „laesst nur eine von zwei parallelen
.select('id')                  // Zustellungen gewinnen; nur der Gewinner bucht"
```

`fulfillRentalPayment` hatte das nicht:

```
await supabase.from('rental_bookings')
  .update({ payment_status: 'paid', status: 'confirmed', ... })
  .eq('id', rentalId) // kein neq, kein select, kein Ergebnis-Check
```

Der Lesecheck weiter oben (`if (rental.payment_status === 'paid') return`) ist eine **Momentaufnahme**: zwischen diesem `SELECT` und dem `UPDATE` passt eine zweite Zustellung. Stripe stellt Webhooks ausdrücklich mehr als einmal zu und wiederholt nach einem Timeout — und was hinter diesem `UPDATE` noch kommt (Payment-Zeile, Plattform-Transaktion, Provisionsberechnung, Audit-Log, zwei Benachrichtigungen), braucht genug Zeit, damit dieser Timeout eintritt.

Ohne Claim gewinnen beide Zustellungen:

- **zwei Zeilen in `payments`** für eine Miete. `payments` ist die Quelle jeder Umsatzzahl in `/api/admin/mis`, `/api/admin/kpi` und `/api/investor` — dort steht der Mietumsatz dann doppelt.
- zwei Einträge in `audit_logs`, zwei Benachrichtigungspaare an beide Seiten.
- ein zweiter Versuch auf `platform_transactions`. Der wird vom partiellen Unique-Index `uq_pltx_rental_succeeded` geblockt — aber nur, wenn dieser Index live existiert. Er stammt aus Migration `20260705_rental_booking_constraints`, und ob die angewendet wurde, ist ohne DB-Zugang nicht prüfbar (siehe „Wahrheitsstand“).

Fix

Derselbe Claim wie in den beiden anderen Zweigen, plus ein Ergebnis-Check: wer das Rennen verliert, kehrt vor der Payment-Zeile um. Dazu im `async_payment_failed`-Zweig ein `.neq('payment_status', 'paid')` für die Miete — der Status-Guard allein trägt nur, solange der Erfolgspfad `status` und `payment_status` gemeinsam setzt.

Gegenprobe im Test: der Nachbau der Datenbank ist sequentiell; ohne eine Stelle, an der ein Test das Fenster zwischen Lesen und Schreiben betreten kann, schreiben `.eq(id)` und `.eq(id).neq(status)` dasselbe. Der Harness hat dafür einen `raceBefore(table, op, effect)`-Haken bekommen (dritte Erweiterung dieser Art nach `rpc()` in Track 20 und `auth.getUser` in Track 21).

CM22-03: `rental_equipment` ist mit dem öffentlichen ANON-Key vollständig lesbar (MITTEL)

Beweis

Live-Sonde am 2026-08-28, ausschließlich mit dem `NEXT_PUBLIC_SUPABASE_ANON_KEY` — also mit einem Schlüssel, der in jedem Browser-Bundle steht:

```
GET /rest/v1/rental_equipment?select=* → 200, 5 Zeilen
GET /rest/v1/rental_bookings?select=* → 401 permission denied
GET /rest/v1/rental_requests?select=* → 401 permission denied
GET /rest/v1/payout_accounts?select=* → 401 permission denied
```

Die Policy stammt aus `20260819_rls_close_gaps_v2.sql` und lautet `FOR SELECT TO anon, authenticated USING (true)`. Ihre Begründung steht im Kopf derselben Datei:

Der Browser-Client (`src/lib/supabase.ts`, `Anon-Key`) liest NUR `rental_equipment`, `salons` und `rental_bookings` — `rental_equipment` behält deshalb unten bewusst seine öffentliche Lesepolicy.

Das stimmt seit Track 7 nicht mehr. **Genau weil** `salons` für `anon` an `permission denied for function is_admin_or_super` scheitert, wurde die Inseratssuche damals auf `/api/rental-listings` umgestellt — serverseitig, mit dem Service-Client. Heute importiert eine einzige Datei den ANON-Client (`src/app/(public)/konto/page.tsx`) und benutzt ihn ausschließlich für `supabase.auth.*`.

Was `USING (true)` offenlegt, ist mehr als ein öffentlicher Katalog:

- **Unveröffentlichte Entwürfe.** `ensurePrimaryListing()` legt für jeden Vermieter, der den Inserats-Editor öffnet, eine Zeile mit `is_available = false` an. Jeder Preis, den er dort einträgt, bevor er online geht, steht ab dem Speichern öffentlich unter `/rest/v1`.
- **Inserate gesperrter und nie freigeschalteter Anbieter.** Track 15 hat `salons.is_active` auf die Geldstrecken gelegt, Track 20 auf die öffentliche Salonseite (404). `/api/rental-listings` filtert entsprechend — die Tabelle selbst filtert nichts. Der Riegel war über PostgREST in einem einzigen Request zu umgehen.

Heute stehen dort fünf Zeilen, alle aus der Demo-Schicht. Der Befund beginnt mit dem ersten echten Vermieter, ohne dass jemand etwas an den Rechten ändert — dieselbe Form wie `newsletter_sends` in Track 20.

Fix — und was er nicht ist

Die Migration nimmt `anon` und `authenticated` den Tabellenzugriff und entfernt die Policy (eine Policy ohne GRANT wäre Dekoration, die etwas anderes suggeriert als sie tut).

Eine **engere Policy** wäre der naheliegendere Weg und funktioniert hier nicht: die Bedingung „Salon ist freigeschaltet“ steht in `salons`, und eine Policy-Unterabfrage dorthin läuft für `anon` in genau das `is_admin_or_super`-Recht, an dem schon der ursprüngliche Join gescheitert ist. Den Sperrzustand nach `rental_equipment` zu denormalisieren wäre ein zweiter Wahrheitsort für dieselbe Angabe. Die öffentliche Sicht hat mit `/api/rental-listings` und `/api/rental-equipment/[id]` bereits zwei Routen, die beide Filter korrekt anwenden — die Tabelle direkt lesbar zu lassen umgeht nur sie.

Dieser Befund ist NICHT behoben. Die Migration ist committet, nicht angewendet; es gibt in diesem Projekt keinen Migrations-Runner und für Agents keinen DB-Zugang. Sie braucht einen Lauf im Supabase-SQL-Editor.

CM22-04: Unbezahlte Reservierungen zählten als Umsatz (MITTEL)

Beweis

`/api/me/rental-revenue` markiert jede Buchung mit einem Flag, an dem die Oberfläche alles weitere aufhängt:

```
const NON_REVENUE_STATUSES = new Set(['cancelled', 'canceled', 'declined', 'rejected', 'refund', ...
countsAsRevenue: !NON_REVENUE_STATUSES.has(status.toLowerCase())
```


`pending` steht auf keiner Ausschlussliste. Eine Buchung ohne jede Zahlung zählte damit als Einnahme — und `/vermieter/mein-inserat/umsatz` filtert genau auf dieses Feld:

```
bookings: res.bookings.filter((b) => b.countsAsRevenue && b.startDate && b.endDate)
```

Daraus entstehen dort: „Einnahmen gesamt“, die Monatsbalken, die Auslastung und eine Preisempfehlung („Senke deinen Tagessatz um 10 %“). Der Leerzustand derselben Seite verspricht dabei wörtlich: „Sobald die erste Buchung **bezahlt** ist, erscheinen hier echte Zahlen.“

Solche Zeilen anzulegen kostet nichts. `POST /api/rental-bookings` schreibt `status: 'pending' / payment_status: 'unpaid'`, **bevor** Stripe überhaupt gefragt wird. Bricht man den Checkout ab, bleibt die Zeile stehen, bis der Cleanup im nächtlichen Payout-Cron sie nach 24 Stunden storniert — bis zu rund 28 Stunden. Ein eigenes Limit hat die Route nicht.

Das ist derselbe Fehlertyp, den die Tracks 8–10 als „erfundene Zahlen“ abgeräumt haben, nur diesmal nicht aus einem Demo-Fallback, sondern aus einer zu weiten Definition — und diesmal von außen steuerbar.

Fix

`countsAsRevenue` verlangt zusätzlich einen Zahlungsstatus aus einer **Positivliste** (`paid`, `succeeded`). Bewusst eine Positivliste: ein neuer Zahlungsstatus soll nicht dadurch zu Umsatz werden, dass niemand daran gedacht hat, ihn auszuschließen. Die Zeile selbst bleibt in der Antwort — der Vermieter soll seine offenen Reservierungen sehen, sie zählen nur nicht als Geld, das angekommen ist.

CM22-05: Mietanfragen ohne eigenes Limit (MITTEL)

Beweis

`POST /api/rental-requests` legt eine Anfrage an, schreibt dem Vermieter eine In-App-Benachrichtigung **und** schickt ihm eine E-Mail mit bis zu 400 Zeichen frei wählbarem Text (`MESSAGE_EXCERPT_LIMIT`). Die Route hatte kein eigenes Limit.

Zwei vorhandene Riegel beantworten eine andere Frage:

- Der **Dedupe-Claim** (Track 5) verhindert, dass *derselbe Inhalt* zweimal ankommt. Ein Zeichen mehr in `message` ergibt einen neuen Fingerprint, eine neue Zeile, eine neue Benachrichtigung, eine neue Mail.
- Die **Idempotenz des Mailversands** hängt an `rental_requests.id`. Jede neue Anfrage hat eine neue.

Übrig blieb das Middleware-Limit von 60 Requests pro Minute und IP — also bis zu **86.400 Mails pro Tag** an ein einzelnes Vermieter-Postfach, von einem einzigen angemeldeten Konto aus, mit der Absenderreputation von ChairMatch. Dieselbe Lücke hat Track 20 beim SMS-Versand geschlossen.

Fix

Zwei Deckel, weil einer die falsche Seite schützt: **20 Anfragen pro Stunde je Konto** begrenzt, wie viel einer versendet; **10 pro Stunde je Mietobjekt** begrenzt, wie viel ein Vermieter abbekommt (sonst

verteilt man die Anfragen über mehrere Konten auf dasselbe Ziel). Beide greifen **vor** dem Datenbank-Lookup: wer fremde Objekt-IDs durchprobiert, verbraucht damit sein eigenes Budget.

CM22-06: UTC statt Berlin, an zwei Stellen (NIEDRIG)

Track 18 hat den Vergangenheits-Vergleich in `/api/rental-bookings` auf `berlinToday()` umgestellt, weil `new Date().toISOString().slice(0, 10)` zwischen 00:00 und 02:00 Berliner Zeit noch den Vortag liefert. Zwei Stellen derselben Domäne standen unverändert weiter:

- `/api/rental-requests` ließ in diesem Fenster eine Anfrage für **gestern** durch (im Sommer zwei Stunden lang, im Winter eine).
- `/api/cron/rental-payouts` vergleicht `rental.start_date > today`. Der Cron läuft um 04:00 Berliner Zeit, da stimmen beide Tage noch überein — aber die Auszahlung hängt damit an der **Startzeit des Crons** statt an der Miete. Ein Verschieben auf 01:00 Berliner Zeit (Sommerzeit: 23:00 UTC des Vortags) hätte jede am selben Tag beginnende Miete um 24 Stunden verspätet, ohne dass sich am Code etwas ändert.

Beide auf `berlinToday()` umgestellt.

CM22-07: Datumsangaben wurden nur auf ihre Form geprüft (NIEDRIG)

Beweis

Die Zod-Regex `/^d{4}-d{2}-d{2}$/` ist ein Formtest, kein Datumstest. `2026-02-30` und `2026-13-45` haben genau diese Form. Was danach passiert, ist in beiden Fällen falsch, auf zwei verschiedene Arten:

`2026-13-45` ergibt `Invalid Date`, die Tagesdifferenz wird `NaN` — und `NaN` ist der stille Fall:

```
const days = rentalDays(startDate, endDate)    // NaN
if (days > 366) ...                            // NaN > 366 → false
const totalCents = computeTotalCents(days, ...) // NaN
if (totalCents <= 0) ...                       // NaN <= 0 → false
```

Beide Riegel lassen den Wert durch. `total_cents: NaN` serialisiert im JSON-Body als `null`, und der Fehler fällt erst in der Datenbank auf — als 500 für eine reine Eingabefehlermeldung.

`2026-02-30` rollt in JavaScript still auf den 2. März weiter (nachgerechnet: `rentalDays('2026-02-30', '2026-03-02')` ergibt `1`). Die Mietdauer wird also für einen anderen Zeitraum gerechnet als den, der gleich in die Datenbank geht — und Postgres weist den 30. Februar dann als `22008` zurück, nachdem der Preis längst feststand.

Fix

Neuer Baustein `src/lib/iso-date.ts` in der Reihe von `uuid.ts`, `csv.ts`, `safe-url.ts` und `content-disposition.ts`: `isCalendarDate()` schreibt den geparkten Tag wieder aus und vergleicht ihn mit der Eingabe — was den Weg hin und zurück überlebt, gibt es wirklich. `inclusiveDayCount()`

wirft bei einem unmöglichen Datum, statt `NaN` zurückzugeben, das jeden nachfolgenden Vergleich still passieren lässt. `/api/rental-bookings` antwortet jetzt mit 400, bevor irgendetwas geschrieben oder eine Stripe-Session erzeugt wird.

Migration

`supabase/migrations/20260828_miet-marktplatz_haertung.sql` — **committet, NICHT angewendet.**

Drei Abschnitte:

1. **`publish_review_pair()`** polymorph (CM22-01, zweite Schicht). Zusätzlich ersetzt `NOW()` – `v_booking_ended_at >= INTERVAL '14 days'` das `EXTRACT(DAY FROM ...)`: das war für die Differenz zweier Zeitstempel richtig, hing aber an einer Eigenschaft des Ausdrucks statt an der Absicht.
2. **REVOKE auf `rental_equipment`** (CM22-03).
3. **Sieben CHECK-Constraints**, die Regeln festschreiben, die heute nur im Route-Handler stehen:

| Tabelle | Constraint | Warum | |---|---|---| | `rental_bookings` | `end_date >= start_date` | Der EXCLUDE-Constraint aus 20260705 fängt das nur für aktive Buchungen ab; eine stornierte mit verdrehtem Zeitraum geht durch | | `rental_bookings` | `total_cents >= 0` | — | | `rental_bookings` | `payment_status` aus fünf Werten | Der Buchungsstatus hat seinen CHECK seit 20260705, der Zahlungsstatus nicht — dabei entscheidet er über Auszahlung, Erstattung und (seit CM22-04) über den Umsatz | | `rental_equipment` | `type` aus vier Werten | Der Typ bestimmt den Provisionssatz (`opraum` → 8 %, sonst 10 %); ein unbekannter Wert fällt still in den 10-%-Zweig | | `rental_equipment` | keine negativen Preise | — | | `rental_equipment` | online nur mit Tagespreis | Steht dreimal im Code (POST `/api/rental-equipment`, PATCH `/api/rental-equipment/[id]`, PATCH `/api/me/listing`), in der Datenbank nirgends | | `rental_equipment` | `available_to > available_from` | — |

Jeder Constraint zählt vorher, wie viele Zeilen ihn heute verletzen, und **bricht mit einer klaren Meldung ab**, statt Zeilen stillschweigend zu verändern — was da liegt, ist eine fachliche Frage. Gegen die heute live sichtbaren Daten (Sonde 2026-08-28: fünf Inseerate, alle vier Typen vertreten, alle mit Tagespreis) greift keiner der Abbrüche.

Ohne Befund

Geprüft und in Ordnung:

- **Die Preisquelle.** POST `/api/rental-bookings` berechnet den Betrag ausnahmslos serverseitig aus `rental_equipment`; der Client schickt nur `equipmentId` und Zeitraum. Kein Feld des Requests erreicht `total_cents`. Dasselbe für die Kostenschätzung in `/api/rental-requests`.
- **Die Provision.** `calculateCommission()` rechnet aus dem Objekttyp, nicht aus dem Request. `createRentalCheckout` setzt kein `allow_promotion_codes` — Rabattbetrag und Anbieteranteil können deshalb nicht auseinanderlaufen.
- **Der Escrow.** Der Payout-Cron zahlt erst ab dem Mietbeginn, prüft `rental_bookings.status / payment_status`, holt die Charge mit `expand: ['latest_charge']` und setzt bei Teilerstattung oder Rückbuchung aus (Track 16). Mehrere Connect-Konten für einen Anbieter führen zum Aussetzen, nicht zum Raten. Der Transfer trägt einen Idempotency-Key.

- **Die Mandantentrennung der Vermieter-Routen.** `requireOwnedSalon()` und `requireOwnedEquipment()` lösen den Besitz über `salons.owner_id` aus der Session auf, nie aus dem Request. `/api/me/rental-revenue` filtert über die eigenen Salons; `rental_bookings` hat live keine `salon_id`, der Bezug läuft ausschließlich über die Mietobjekte, und das ist überall so umgesetzt.
- **Die Berechtigung im Miet-Storno.** Mieter, Vermieter und Admin werden aus der echten Beziehung abgeleitet, nicht aus „ist nicht fremd“; eine bereits ausgezahlte Buchung wird abgelehnt statt aus dem Plattformguthaben erstattet; eine gescheiterte Erstattung storniert nicht.
- **Die Statuswechsel der Anfrage.** `PATCH /api/rental-requests/[id]` prüft Beteiligung **und** Richtung (nur der Anfragende zieht zurück, nur der Empfänger sagt zu oder ab) und lässt nur `open` als Ausgangszustand zu.
- **Die Bewertungs-Berechtigung.** `/api/reviews/rental` leitet die Richtung aus der Buchung ab, verlangt einen abgeschlossenen Zeitraum und weist Unbeteiligte mit 403 ab. Der Unique-Index verhindert Doppelbewertungen.
- **Die Anbieter-Sperre.** `salonAcceptsBusiness()` liegt auf Buchung, Anfrage und Inseratsdetail; die Inseratsliste filtert bewusst nur bei einem ausdrücklichen `false` (Track 15).
- **Die Auszahlungsdaten.** Die volle IBAN verlässt den Server nie wieder; GET liefert nur die letzten vier Stellen; die IBAN wird gegen ihre Prüfziffern validiert. `payout_accounts` ist für `anon` live gesperrt (Sonde bestätigt).
- **Die anon-Exposition der übrigen Miet-Tabellen.** `rental_bookings`, `rental_requests`, `rental_request_dedupe`, `platform_transactions`, `provider_stripe_accounts`, `commissions`, `tenant_profiles` antworten alle mit 42501. Die Lockdown-Migration aus Track 8 wirkt live.
- **Zombie-Buchungen.** Der `checkout.session.expired`-Webhook gibt eine nicht bezahlte Buchung nach 30 Minuten frei, der Cron nach 24 Stunden als Rückfall — die pending-Blockade des Vermieterkalenders ist also gedeckelt. Ohne den CM22-04-Fix war sie trotzdem in der Umsatzanzeige zu sehen.

Tests

18 neue Tests in `src/__tests__/track-22-miet-marktplatz.test.ts`, verteilt auf fünf Blöcke: Webhook-Idempotenz (3), Umsatzabgrenzung (3), Anfrage-Limit und Berliner Kalendertag (3), Kalendertage (5), Bewertungs-Freischaltung (4).

Zwei angepasste Bestandstests in der Track-20-Datei. Sie prüften „der Cron zählt nur echte Freischaltungen“ über den RPC-Aufruf — den es nicht mehr gibt. Die Zusage ist unverändert geblieben, sie läuft jetzt über das UPDATE. Der Erfolgsfall wurde dabei **strenger**: er prüft nun, dass die Zeile wirklich auf `published = true` steht — das konnte der alte Test nicht, weil der RPC-Nachbau nichts geschrieben hat.

Zwei Bestandsdateien (`rental-requests/__tests__/dedupe.test.ts` und `route.e2e.test.ts`) haben `__resetRateLimits()` im `beforeEach` bekommen: sie schicken bewusst viele Anfragen auf dasselbe Mietobjekt und liefern sonst in das neue 429 statt in den Pfad, den sie prüfen. Keine Abdeckung entfernt.

Harness-Erweiterung: `raceBefore(table, op, effect)` im `fake-supabase`-Nachbau. Ohne eine Stelle, an der ein Test das Fenster zwischen `SELECT` und `UPDATE` betreten kann, lässt sich ein Compare-and-Swap nicht von einem gewöhnlichen Update unterscheiden — beide schreiben in einem sequentiellen Nachbau dasselbe.

Nachtrag ohne eigenen Befund: `track-21-session-cookie-praefix.test.ts` ist im vollständigen Lauf über 77 parallele Dateien zeitweise in das 5-Sekunden-Zeitlimit gelaufen (der erste Test der Datei zieht den gesamten `auth.config`-Graph kalt herein; einzeln läuft er unter einer Sekunde). Nur die Frist dieses einen Tests wurde auf 30 Sekunden gesetzt, keine Zusage geändert.

Gegenprobe

Mit zurückgesetztem Produktivcode (sechs Dateien auf `2737dde`) fallen **10 von 20** neuen und angepassten Tests durch:

Test	Befund
verliert das Rennen, wenn die Zahlung zwischen Lesen und Schreiben schon verbucht wurde	CM22-02
markiert eine unbezahlte Reservierung NICHT als Umsatz	CM22-04
stoppt die Flut, obwohl jede Nachricht einen neuen Fingerprint hat	CM22-05
weist ein Datum ab, das in Berlin bereits vergangen ist	CM22-06
weist eine Buchung mit unmöglichem Datum mit 400 ab, nicht mit 500	CM22-07
schaltet eine überfällige einseitige Miet-Bewertung frei	CM22-01
schaltet dieselbe Bewertung beim zweiten Lauf nicht noch einmal frei	CM22-01
ruft <code>publish_review_pair()</code> nicht mehr	CM22-01
zählt eine echte Freischaltung als veröffentlicht (Track 20, angepasst)	CM22-01
zählt eine fehlgeschlagene Freischaltung nicht als veröffentlicht (Track 20, angepasst)	CM22-01

Die übrigen zehn bleiben grün, weil sie Verhalten prüfen, das schon vorher richtig war (erste Zustellung, bezahlte Buchung, nicht fällige Bewertung) oder weil sie die neue, nicht zurücksetzbare Datei `src/lib/iso-date.ts` direkt als Funktion prüfen.

Operative Folgen des Deploys

1. **Die Umsatzanzeige des Vermieters wird kleiner.** Offene, unbezahlte Reservierungen zählen nicht mehr als Einnahme — das gilt auch für Zeilen, die heute schon dort stehen. Die Zahlen davor waren zu hoch, nicht die jetzigen zu niedrig. 2. **Einseitige Miet-Bewertungen werden erstmals sichtbar.** Der nächste Cron-Lauf (03:30) schaltet alles frei, was älter als 14 Tage ist und bisher liegen blieb — je nach Bestand auf einmal. Damit füllen sich auch `avg_rating_as_tenant` und `avg_rating_as_provider` zum ersten Mal. 3. **Mietanfragen sind gedeckelt.** 20 pro Stunde je Konto, 10 je Mietobjekt. Für normale Nutzung unauffällig. 4. **Unmögliche Datumsangaben antworten mit 400 statt 500.** Kein bestehendes Formular schickt solche Werte.

Offen und ausdrücklich NICHT als behoben ausgegeben

- **CM22-03** (`rental_equipment` anon-lesbar) braucht einen Lauf der Migration im Supabase-SQL-Editor. Bis dahin ist die Tabelle offen — nachprüfbar mit `./scripts/rls-anon-probe.sh`.
- **Die sieben CHECK-Constraints** und die reparierte `publish_review_pair()`-Funktion liegen in derselben Migration und teilen diesen Zustand. Für CM22-01 wirkt der Anwendungs-Fix im Cron auch ohne sie.
- **Ob `uq_pltx_rental_succeeded` und `rental_bookings_no_overlap` live existieren**, ist von hier aus nicht prüfbar. Beide stammen aus `20260705_rental_booking_constraints.sql`; der Code verlässt sich an zwei Stellen auf sie (Doppel-Transaktions-Backstop, Doppelbuchungs-Riegel).
- **Die live angewendeten RLS-Policies** für die Rolle `authenticated` bleiben ungeprüft — dafür wäre eine echte Anmeldung nötig. Unverändert gegenüber Track 21.

Wahrheitsstand

Aussage	Stand
IMPLEMENTIERT	teilweise — sechs von sieben Befunden im Code behoben; CM22-03 ausschließlich als committete Migration
GETESTET	ja — 18 neue Tests, 1526 gesamt grün, Gegenprobe 10/20 rot ohne Fix
TYPECHECK	ja — <code>tsc --noEmit</code> ohne Fehler
LINT	ja — 0 Fehler, 21 Warnungen, keine davon in einer Datei dieses Tracks
MIGRATION ANGEWENDET	nein — committet, nicht ausgeführt. Es gibt keinen Migrations-Runner und für Agents keinen DB-Zugang
DEPLOYED	nach <code>./deploy.sh</code> — Vercel baut automatisch
LIVE_VERIFIZIERT	teilweise. Die anon-Exposition (CM22-03) und das Live-Schema von <code>rental_equipment</code> wurden am 2026-08-28 gegen die Produktionsdatenbank gesendet (read-only, nur ANON-Key). Alles andere ist gegen den Nachbau geprüft, nicht gegen die Produktion — kein Agent-Zugang zur Produktionsdatenbank, keine Möglichkeit, sich live anzumelden.